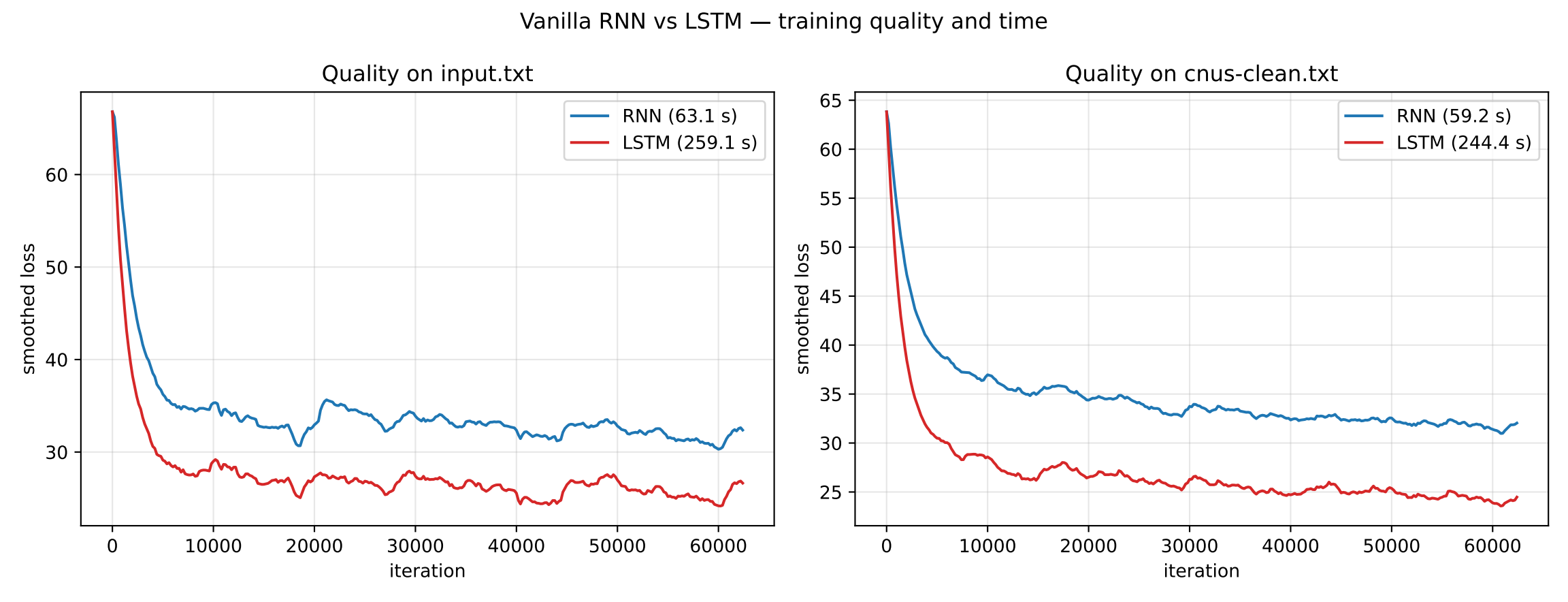
Лабораторная работа № 3 — сводные результаты

Рекуррентные сети: Vanilla RNN / LSTM (NumPy) и LSTM (PyTorch)

run	dataset	iter	seconds	final loss
rnn_input	input.txt	62500	1:03.1	32.371
rnn_cnus	cnus-clean.txt	62500	59.2 s	32.039
lstm_input	input.txt	62500	4:19.1	26.640
lstm_cnus	cnus-clean.txt	62500	4:04.4	24.477

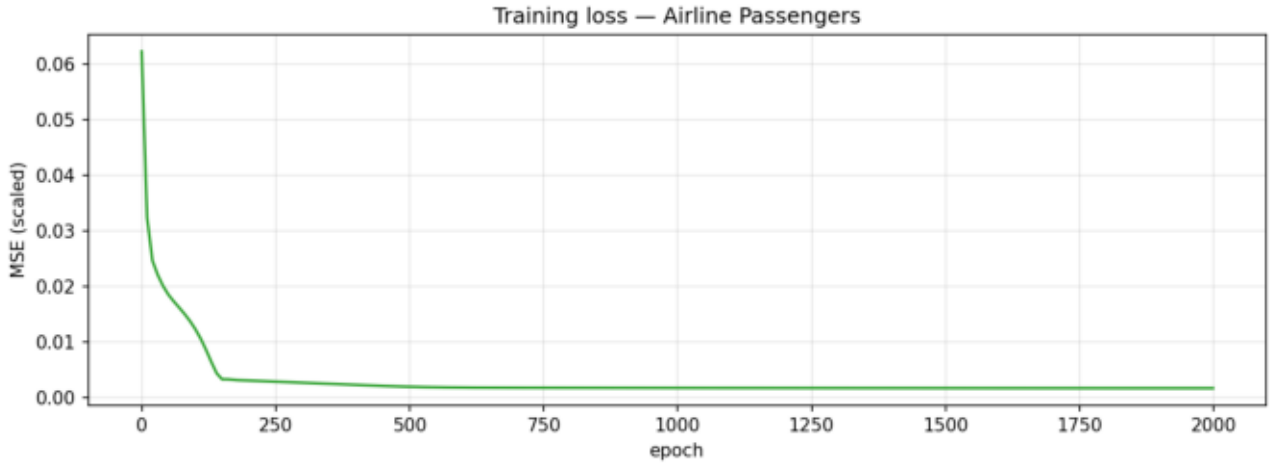
PyTorch LSTM runs:

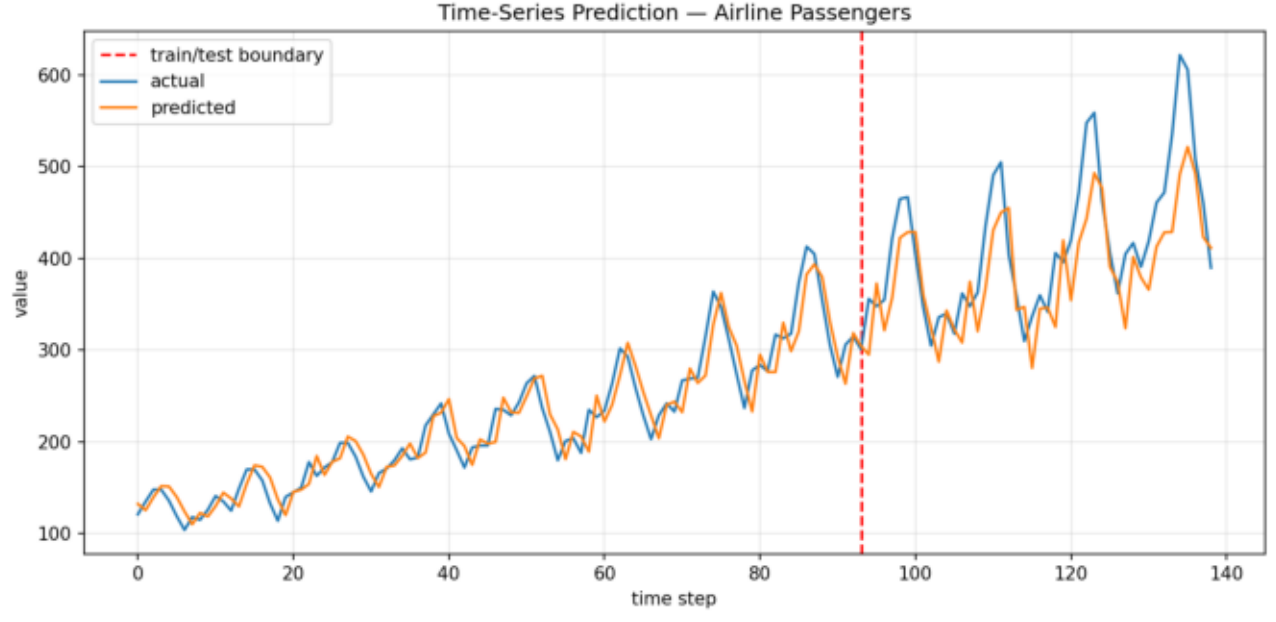
```
airline (airline-passengers.csv): epochs=2000, train time=3.6 s
    RMSE (original units): train=21.338, test=51.524
shampoo (shampoo.csv): epochs=2000, train time=3.4 s
    RMSE (original units): train=24.206, test=246.207
```


Vanilla RNN vs LSTM — training quality and time
Quality on input.txt
RNN (63.1 s)
LSTM (259.1 s)
smoothed loss
iteration
Quality on cnus-clean.txt
RNN (59.2 s)
LSTM (244.4 s)
smoothed loss
iteration

Временной ряд: Airline Passengers (обучение 3.6 s)

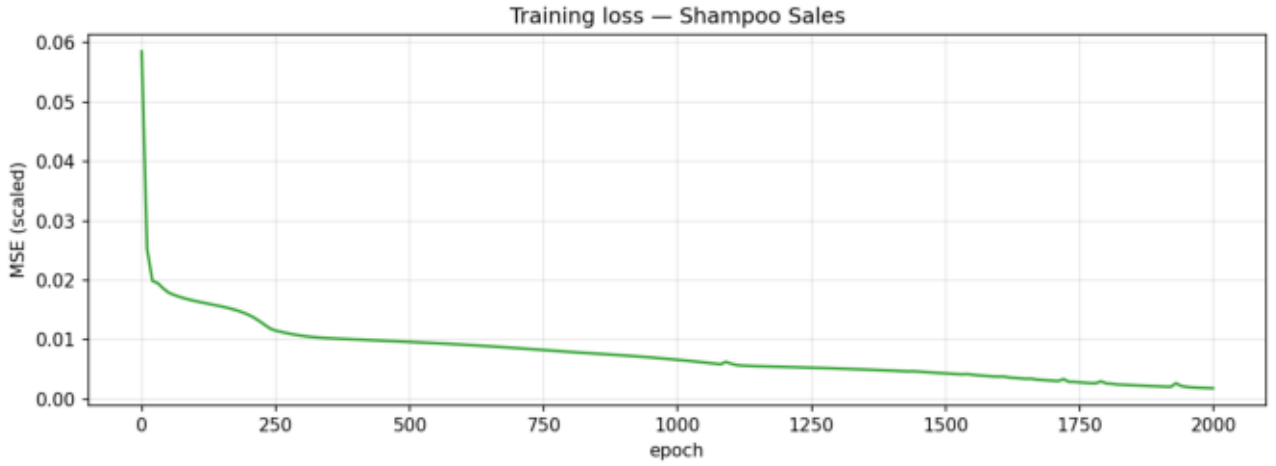
MSE по эпохам

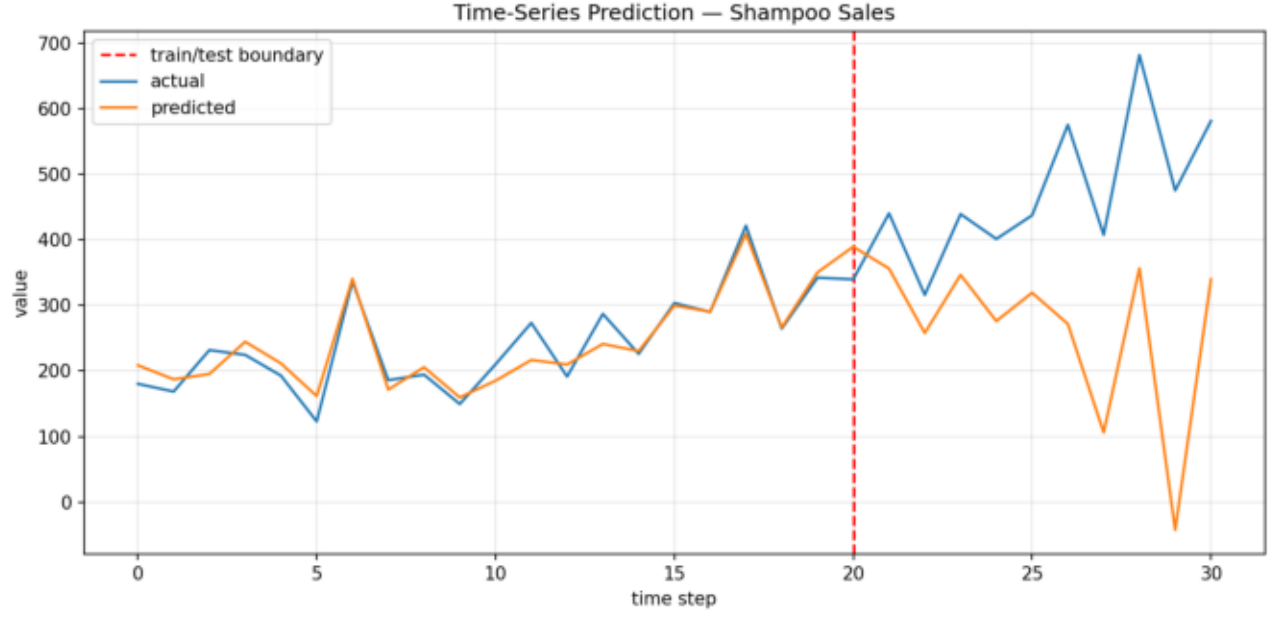




Временной ряд: Shampoo Sales (обучение 3.4 s)

MSE по эпохам





Выводы

RNN input.txt : 63.1 s, 62500 ит., итоговый loss 32.371

RNN cnus-clean.txt: 59.2 s, 62500 ит., итоговый loss 32.039

LSTM input.txt : 259.1 s, 62500 ит., итоговый loss 26.640

LSTM cnus-clean.txt: 244.4 s, 62500 ит., итоговый loss 24.477

input.txt: LSTM медленнее RNN в 4.1 раза (259.1 s vs 63.1 s).

cnus-clean.txt: LSTM медленнее RNN в 4.1 раза (244.4 s vs 59.2 s).

По кривым видно, что LSTM снижает loss быстрее и глубже: аддитивное обновление памяти c(t) сохраняет градиент на длинных последовательностях, тогда как RNN страдает от его затухания.

Airline Passengers: RMSE train 21.34, test 51.52 (исходные единицы), обучение 3.6 s.

Shampoo Sales: RMSE train 24.21, test 246.21 (исходные единицы), обучение 3.4 s.

PyTorch-LSTM уверенно воспроизводит ряд на train-части; на test-части ошибка выше, что ожидаемо для такой маленькой сети (hidden_size=2).